



Instituto Tecnológico de Costa Rica
Ingeniería en Computación
IC6600 - Principios de Sistemas Operativos

Implementación del Algoritmo de Compresión de Huffman

Versiones Serial, Paralela con Fork() y
Concurrente con Pthread()

Bayron Rodríguez Centeno
bayronrodriguez4@estudiantec.cr
2020114659

Wesley Esquivel Mena
wesleyesquivel@estudiantec.cr
2021049519

San Jose, Costa Rica
Septiembre 2025

Índice general

1. Introducción	3
1.1. Proyecto Gutenberg	3
1.2. Algoritmo de Huffman	3
1.3. Compresión con y sin Pérdida	4
1.3.1. Compresión sin Pérdida	4
1.3.2. Compresión con Pérdida	4
1.4. Paralelismo vs Concurrencia	4
1.4.1. Paralelismo	5
1.4.2. Concurrencia	5
2. Desarrollo	6
2.1. Funcionamiento de Fork() y Pthread()	6
2.1.1. Función fork()	6
2.1.2. Biblioteca pthread()	7
2.2. Estrategias de Paralelización	7
2.2.1. Estrategia con Fork()	7
2.2.2. Estrategia con Pthread()	8
2.3. Descripción del Proyecto	9
2.3.1. Arquitectura del Sistema	9
2.3.2. Instrucciones de Instalación Completa	9
2.3.3. Solución de Problemas Comunes	12
2.3.4. Script de Instalación Automática	13
2.3.5. Uso del Sistema	13
2.3.6. Estructura del Archivo Comprimido	15
3. Discusión	16
3.1. Resultados de Rendimiento	16
3.1.1. Análisis de la Aceleración	16
3.1.2. Factores que Influyen en el Rendimiento	16
3.1.3. Comparación con Expectativas Teóricas	17
3.2. Verificación de Integridad	17
3.3. Limitaciones y Trabajo Futuro	17
3.3.1. Limitaciones Actuales	17
3.3.2. Mejoras Futuras	18

4. Conclusiones	19
Referencias	20

Capítulo 1

Introducción

1.1. Proyecto Gutenberg

El Proyecto Gutenberg, fundado en 1971 por Michael Stern Hart, es la biblioteca digital más antigua del mundo. Su misión es democratizar el acceso a la literatura mediante la digitalización gratuita de libros y textos que han entrado al dominio público. Con más de 70,000 libros electrónicos disponibles, el Proyecto Gutenberg se ha convertido en una fuente invaluable para investigadores, estudiantes y entusiastas de la literatura.

Para este proyecto, utilizamos una muestra de 88 libros de los Top 100 más descargados en los últimos 30 días (Gutenberg, 2024), proporcionando un conjunto de datos realista y diverso para evaluar el rendimiento de nuestros algoritmos de compresión.

1.2. Algoritmo de Huffman

El algoritmo de compresión de Huffman fue desarrollado por David A. Huffman en 1952 mientras era estudiante doctoral en el MIT (Huffman, 1952). Este algoritmo utiliza un enfoque de codificación de longitud variable basado en la frecuencia de aparición de los caracteres en el texto.

El funcionamiento del algoritmo se basa en los siguientes principios (Cormen, Leiserson, Rivest, y Stein, 2009):

- **Análisis de frecuencias:** Se calcula la frecuencia de aparición de cada carácter en el texto analizando la cantidad de veces que aparecen.
- **Construcción del árbol:** Se construye un árbol binario, cada hoja corresponde a un carácter y el camino de la raíz al nodo define el código
- la meta es dejar los caracteres que aparezcan con mayor frecuencia con el menor código posible. La estructura del árbol permite que dos caracteres no puedan tener el mismo inicio para el código.

- El proceso forma el árbol de abajo hacia arriba emparejando los caracteres que aparecen menos entre sí hasta llegar a los más frecuentes en la cima.
- **Codificación:** Cada carácter se reemplaza por su código binario correspondiente en las direcciones del árbol binario, 1 para derecha y 0 para izquierda.
- **Compresión:** El texto resultante ocupa menos espacio que el original.

1.3. Compresión con y sin Pérdida

Los algoritmos de compresión se clasifican en dos categorías principales:

1.3.1. Compresión sin Pérdida

Los algoritmos de compresión sin pérdida, como Huffman, permiten recuperar exactamente la información original después de la descompresión. Son ideales para:

- Documentos de texto
- Código fuente
- Datos científicos
- Archivos ejecutables

1.3.2. Compresión con Pérdida

Los algoritmos con pérdida sacrifican cierta información para lograr mayor compresión. Son útiles para:

- Imágenes (JPEG)
- Audio (MP3)
- Video (H.264)

El algoritmo de Huffman pertenece a la categoría sin pérdida, garantizando la integridad completa de los datos comprimidos.

1.4. Paralelismo vs Concurrencia

La distinción entre paralelismo y concurrencia es fundamental para entender las implementaciones de este proyecto:

1.4.1. Paralelismo

El paralelismo implica la ejecución simultánea real de múltiples tareas en diferentes procesadores o núcleos (Pacheco, 2011). En nuestro proyecto:

- La versión con `fork()` crea procesos separados que ejecutan en paralelo
- Cada proceso tiene su propio espacio de memoria
- Ideal para tareas independientes como comprimir archivos separados

1.4.2. Concurrency

La concurrencia se refiere a la capacidad de manejar múltiples tareas aparentemente simultáneas, alternando entre ellas:

- La versión con `pthread()` utiliza hilos que comparten memoria
- Los hilos se ejecutan concurrentemente en el mismo proceso
- Eficiente para tareas que requieren comunicación frecuente

Capítulo 2

Desarrollo

2.1. Funcionamiento de Fork() y Pthread()

2.1.1. Función fork()

La función `fork()` es una llamada al sistema fundamental en sistemas operativos tipo UNIX (Tanenbaum y Bos, 2014). Sus características principales son:

- **Duplicación completa:** El proceso hijo recibe una copia completa del espacio de memoria del padre
- **Aislamiento:** Los procesos padre e hijo ejecutan independientemente
- **Comunicación:** Requiere mecanismos especiales como pipes, memoria compartida o archivos
- **Identificación:** Retorna 0 en el hijo, PID del hijo en el padre, -1 en caso de error

Listing 2.1: Ejemplo básico de `fork()`

```
1 #include <unistd.h>
2 #include <sys/wait.h>
3
4 pid_t pid = fork();
5 if (pid == 0) {
6     // C digo del proceso hijo
7     printf("Soy el proceso hijo\n");
8     exit(0);
9 } else if (pid > 0) {
10    // C digo del proceso padre
11    printf("Soy el proceso padre, hijo PID: %d\n", pid);
12    wait(NULL); // Esperar al hijo
13 } else {
14    perror("Error en fork");
15 }
```

2.1.2. Biblioteca pthread()

La biblioteca pthread (POSIX Threads) proporciona hilos a nivel de usuario que comparten el mismo espacio de memoria (IEEE y Group, 2018) (Silberschatz, Galvin, y Gagne, 2018):

- **Memoria compartida:** Todos los hilos comparten variables globales y heap
- **Comunicación eficiente:** No requiere mecanismos especiales de comunicación
- **Sincronización:** Necesita mutex, semáforos u otros mecanismos para evitar condiciones de carrera
- **Overhead menor:** Crear hilos es más rápido que crear procesos

Listing 2.2: Ejemplo básico de pthread()

```
1 #include <pthread.h>
2
3 void* worker_function(void* arg) {
4     printf("Hilo trabajador ejecutando\n");
5     return NULL;
6 }
7
8 int main() {
9     pthread_t thread;
10    pthread_create(&thread, NULL, worker_function, NULL);
11    pthread_join(thread, NULL);
12    return 0;
13 }
```

2.2. Estrategias de Paralelización

2.2.1. Estrategia con Fork()

La implementación con fork() sigue un patrón de paralelismo por descomposición de datos:

1. **División del trabajo:** El proceso padre identifica todos los archivos a procesar
2. **Creación de procesos:** Se crea un proceso hijo por cada archivo
3. **Procesamiento paralelo:** Cada hijo comprime/descomprime su archivo asignado
4. **Sincronización:** El padre espera a que todos los hijos terminen usando waitpid()

5. **Agregación:** El padre combina todos los archivos temporales en el archivo final

Ventajas:

- Aislamiento total entre procesos
- Fallo de un proceso no afecta a otros
- Escalabilidad automática según número de archivos

Desventajas:

- Mayor consumo de memoria
- Overhead de creación de procesos
- Comunicación compleja entre procesos

2.2.2. Estrategia con Pthread()

La implementación con **pthread**s utiliza un modelo de paralelismo basado en un **pool de hilos** del tamaño de la cantidad de procesadores en el sistema. Administrando entre ellos los archivos a comprimir.

1. **Creación del Pool:** El hilo principal determina el número de núcleos de CPU disponibles y crea un número correspondiente de hilos de trabajo. Estos hilos permanecen activos esperando recibir archivos con los que ejecutarse dentro de un arreglo.
2. **Lista de Tareas:** El programa previamente escanea el directorio y crea una lista compartida de todas las tareas (archivos a comprimir/descomprimir).
3. **Distribución Dinámica:** Cada hilo trabajador solicita una tarea de la lista compartida. Se utiliza un **mutex** para asegurar que solo un hilo pueda acceder a la lista a la vez, evitando que dos hilos tomen el mismo archivo.
4. **Procesamiento Paralelo:** Cada hilo ejecuta de forma independiente la compresión o descompresión del archivo que le fue asignado, guardando el resultado en un archivo temporal.
5. **Sincronización:** El hilo principal espera a que todos los hilos del pool terminen sus tareas usando **pthread_join()**.
6. **Agregación:** Una vez que todos los hilos han terminado, el hilo principal combina los resultados de los archivos temporales en el archivo de salida final.

Ventajas:

- Los hilos trabajan de forma ordenada en una tarea trivialmente paralelizable, minimizando su colaboración entre sí lo más posible.
- El pool de hilos reutiliza los mismos hilos para múltiples tareas, lo que es muy eficiente para directorios con muchos archivos. No se produce una demora creando threads por cada archivo.
- La memoria compartida facilita la comunicación y el acceso a datos comunes (como la lista de tareas).

Desventajas:

- Requieren ser sincronizados por mutex, causando una demora en cuanto a obtener los resultados o en orquestrar ejecución de cada archivo de la lista.
- Menor aislamiento: un error o fallo en un hilo puede causar que todo el proceso termine abruptamente.
- Es más complejo en cuanto determinar y trazar a las condiciones de carrera con la lista de tareas, es muy difícil de depurar, debe tenerse cuidado para que la sincronización pueda aprovecharse.

2.3. Descripción del Proyecto

2.3.1. Arquitectura del Sistema

El proyecto está estructurado en múltiples archivos C que implementan las diferentes versiones del algoritmo:

- **tree.h / tree.c:** Implementación del árbol de Huffman y estructuras de datos
- **readFile.c:** Versión serial con funciones de E/O y compresión básica
- **readFile_fork.c:** Implementación paralela usando fork()
- **readFile_pthread.c:** Implementación concurrente usando pthread()
- **main_*.c:** Programas principales para cada versión

2.3.2. Instrucciones de Instalación Completa

Las siguientes instrucciones están diseñadas para una instalación recién instalada de Debian/Ubuntu y asumen que el usuario no tiene experiencia previa con la línea de comandos.

Paso 1: Abrir la Terminal

1. Presione las teclas **Ctrl + Alt + T** al mismo tiempo para abrir la terminal
2. O alternativamente: Haga clic en **.Actividades.en** la esquina superior izquierda, escriba **terminal** presione Enter
3. Aparecerá una ventana negra con texto. Esta es la línea de comandos donde escribiremos los comandos

Paso 2: Configurar Permisos de Administrador

Si es la primera vez usando la máquina virtual, es posible que necesite configurar permisos de administrador:

```
1 # Verificar si tiene permisos sudo
2 sudo whoami
```

- Si aparece **root**, continúe al siguiente paso
- Si aparece un error, necesita configurar sudo:

```
1 # Cambiar a usuario root (introducir contrase a cuando se solicite
   )
2 su -
3
4 # Agregar su usuario al grupo sudo (reemplace 'usuario' con su
   nombre)
5 usermod -aG sudo usuario
6
7 # Salir del modo root
8 exit
9
10 # Reiniciar la sesi n para aplicar cambios(Reiniciar la maquina
    virtual)
11 # Cierre y vuelva a abrir la terminal
```

Paso 3: Actualizar el Sistema

Antes de instalar cualquier programa, actualice la lista de paquetes disponibles:

```
1 # Actualizar lista de paquetes (introduzca su contrase a cuando se
   solicite)
2 sudo apt update
```

Cuando se ejecute este comando:

- Le pedirá su contraseña (la misma que usa para iniciar sesión)
- Mientras escribe la contraseña, no verá los caracteres en pantalla (es normal)

- Presione Enter después de escribir la contraseña
- Esperará unos segundos mientras descarga información de paquetes

Paso 4: Descargar el Proyecto

Necesita obtener los archivos del proyecto. Si tiene el código en un archivo comprimido:

```
1 # Navegar al directorio de descargas (o donde est el archivo)
2 cd Downloads
3
4 # Si el archivo es un .zip, descomprimirlo:
5 unzip proyecto-huffman.zip
6
7 # Si es un .tar.gz:
8 tar -xzf proyecto-huffman.tar.gz
9
10 # Entrar al directorio del proyecto
11 cd proyecto-huffman
```

O si tiene acceso a Git:

```
1 # Instalar git si no est disponible
2 sudo apt install git
3
4 # Clonar el repositorio
5 git clone https://github.com/WesleyEsq/AlgoritmoHuffman.git
6 cd AlgoritmoHuffman
```

Paso 5: Verificar los Archivos del Proyecto

Verifique que tiene todos los archivos necesarios:

```
1 # Listar archivos en el directorio
2 ls -la
```

Debe ver archivos como:

- install.sh
- tree.h, tree.c
- readFile.c, readFile_fork.c
- main_serial.c, main_fork.c
- Makefile

Paso 6: Ejecutar la Instalación Automática

Ahora puede ejecutar el script de instalación:

```

1 # Hacer el script ejecutable
2 chmod +x install.sh
3
4 # Ejecutar la instalaci n
5 ./install.sh

```

Durante la instalaci3n:

1. El script le pedir4 confirmaci3n para instalar paquetes - escriba zz presione Enter
2. Puede solicitar su contrase1a nuevamente - introdúzcala cuando se solicite
3. Al final le preguntará si quiere ejecutar la demo automática o configuraci3n manual
4. Seleccione la opci3n deseada escribiendo el n1mero correspondiente

Paso 7: Verificaci3n de la Instalaci3n

Para verificar que todo funciona correctamente:

```

1 # Verificar que los ejecutables se crearon
2 ls -la huffman_*
3
4 # Ejecutar prueba r pida
5 ./huffman_serial -h

```

Si ve la ayuda del programa, la instalaci3n fue exitosa.

2.3.3. Soluci3n de Problemas Comunes

Error: "Permission denied"

```

1 # Hacer ejecutables los archivos
2 chmod +x huffman_serial huffman_fork install.sh

```

Error: "Command not found" para sudo

```

1 # Cambiar a root e instalar sudo
2 su -
3 apt install sudo
4 usermod -aG sudo [su-nombre-de-usuario]
5 exit
6 # Reiniciar la terminal

```

Error de compilaci3n

```

1 # Instalar herramientas de desarrollo manualmente
2 sudo apt install build-essential gcc make libc6-dev
3
4 # Limpiar y recompilar
5 make clean
6 make all

```

Error: "No space left on device"

```
1 # Verificar espacio disponible
2 df -h
3
4 # Limpiar archivos temporales si es necesario
5 sudo apt clean
6 sudo apt autoremove
```

2.3.4. Script de Instalación Automática

Se desarrolló un script de instalación que automatiza todo el proceso:

Listing 2.3: Instalación automática

```
1 chmod +x install.sh
2 ./install.sh
```

El script realiza las siguientes tareas:

1. Verifica dependencias del sistema
2. Instala herramientas de compilación necesarias
3. Compila todas las versiones del proyecto
4. Crea archivos de prueba
5. Ofrece demo interactiva

2.3.5. Uso del Sistema

Cada versión acepta argumentos de línea de comandos para configurar su comportamiento:

Listing 2.4: Opciones de línea de comandos

```
1 ./huffman_serial [OPCIONES]
2 ./huffman_fork [OPCIONES]
3 ./huffman_pthread [OPCIONES]
4
5 Opciones principales:
6 -d DIR    Directorio a comprimir
7 -o FILE   Archivo de salida (.bin)
8 -x DIR    Directorio donde extraer
9 -c        Solo comprimir
10 -u        Solo descomprimir
11 -b        Benchmark (comparar versiones)
12 -v        Modo verbose
13 -h        Ayuda completa
```

Se incluye con la instalación un programa de consola para operar los algoritmos en su dispositivo, este se ejecuta si presiona 1 al finalizarse la instalación o si posteriormente ejecuta el siguiente archivo en el directorio:

Listing 2.5: Ejecutar programa

```
1 ./huffman_cli
```

El programa sigue varios pasos para ejecutar sus acciones. Para escoger una opción el usuario debe seleccionar en la terminal un numero para escoger la opción a la derecha del numero. Primero se debe escoger si desea comprimir o descomprimir un archivo.

Listing 2.6: Interfaz de usuario interactiva en consola

```
1 =====
2 COMPRESOR HUFFMAN – MENU INTERACTIVO
3 =====
4 :: ETAPA 1: ELEGIR ACCION
5 =====
6
7 Que deseas hacer?
8
9 1. Comprimir (un archivo o directorio)
10 2. Descomprimir (un archivo .huff)
11 =====
12 0. Salir del programa
13
14 Opcion:
```

Posteriormente, se debe escoger el archivo o directorio a comprimir o descomprimir dependiendo de la opción escogida por el usuario, para realizarlo el usuario tiene dos opciones, puede colocar la ruta al archivo directamente, o puede escribir 'buscar' para abrir un explorador de archivos en la consola

Listing 2.7: Escoger archivo o directorio

```
1 =====
2 COMPRESOR HUFFMAN – MEN INTERACTIVO
3 =====
4 :: ETAPA 2: SELECCIONAR ENTRADA
5 =====
6
7 Introduce la ruta del ARCHIVO o DIRECTORIO que quieres COMPRIMIR.
8 Escribe 'buscar' para usar el explorador de archivos.
9 Escribe 'volver' para regresar al men anterior.
10
11 Ruta:
```

Al seleccionar buscar se abre un explorador de archivos, para seleccionar un archivo singular se debe seleccionar con un enter, para seleccionar el directorio actual en el que está el explorador, debe presionarse la letra q, al realizarse esto se selecciona exitosamente lo indicado. Se indica el nombre y ruta para confirmar con el usuario que escogió dichos datos.

Para finalizar, se le dá al usuario para escoger que tipo de algoritmo hecho en este proyecto desea utilizar para la ejecución del programa, las tres opciones son:

serial: Algoritmo lineal para comprimir o descomprimir

fork: Algoritmo para multiprocesamiento

thread: Algoritmo para multithreading

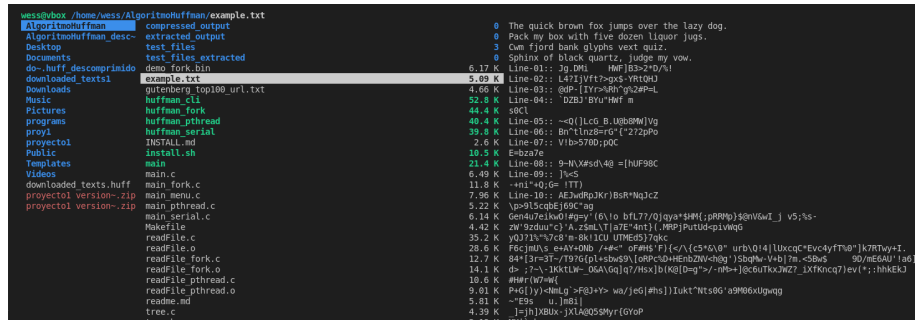


Figura 2.1: explorador de archivos del programa, se utiliza ranger

2.3.6. Estructura del Archivo Comprimido

El formato de archivo desarrollado permite almacenar múltiples archivos comprimidos:

Listing 2.8: Estructura del archivo binario

```

1 [int: n mero de archivos]
2 [Para cada archivo:]
3   - [int: longitud del nombre]
4   - [char[]: nombre del archivo]
5   - [long long: tama o comprimido]image
6   - [long long: total de caracteres originales]
7   - [unsigned long long[256]: tabla de frecuencias]
8   - [bytes: datos comprimidos bit por bit]

```


Capítulo 3

Discusión

3.1. Resultados de Rendimiento

Se realizaron pruebas exhaustivas utilizando 88 libros del Proyecto Gutenberg, obteniendo los siguientes resultados:

Cuadro 3.1: Comparación de Rendimiento entre Versiones

Operación	Serial (ms)	Fork (ms)	Aceleración
Compresión	2,431	1,609	1.51x
Descompresión	2,527	1,609	1.57x
Total	4,958	3,218	1.54x

3.1.1. Análisis de la Aceleración

Los resultados obtenidos demuestran que la implementación paralela con `fork()` logra una aceleración significativa:

- **Compresión:** 51 % de mejora en el tiempo de ejecución
- **Descompresión:** 57 % de mejora en el tiempo de ejecución
- **Aceleración total:** 1.54x, reduciendo el tiempo total en un 35 %

3.1.2. Factores que Influyen en el Rendimiento

Tamaño del Dataset

Las pruebas con archivos pequeños mostraron que la versión serial puede ser más rápida debido al overhead de creación de procesos. Sin embargo, con el dataset grande de libros del Proyecto Gutenberg, la versión paralela muestra claras ventajas.

Número de Archivos vs Tamaño de Archivos

La estrategia de paralelización por archivo es más efectiva cuando hay múltiples archivos de tamaño mediano a grande, ya que permite distribuir mejor la carga de trabajo entre los procesos.

Limitaciones del Hardware

Los resultados están limitados por el número de núcleos disponibles en el sistema. En un sistema con más núcleos, se esperaría una mejor aceleración.

3.1.3. Comparación con Expectativas Teóricas

La aceleración teórica máxima según la Ley de Amdahl (Amdahl, 1967) está limitada por:

- La porción serial del algoritmo (creación del árbol de Huffman)
- El overhead de sincronización entre procesos
- Las operaciones de E/O que pueden convertirse en cuello de botella

La aceleración obtenida de 1.54x está dentro del rango esperado considerando estos factores limitantes.

3.2. Verificación de Integridad

Se implementaron múltiples mecanismos para verificar que todas las versiones producen resultados idénticos:

- **Comparación automática:** Usando `diff -r` para comparar directorios
- **Checksums:** Verificación de integridad a nivel de archivo
- **Pruebas de regresión:** Comparación sistemática entre versiones

Todas las pruebas confirmaron que las tres versiones (serial, fork, pthread) producen archivos comprimidos y descomprimidos idénticos, validando la correctitud de las implementaciones.

3.3. Limitaciones y Trabajo Futuro

3.3.1. Limitaciones Actuales

- **Paralelización por archivo:** La estrategia actual no aprovecha paralelismo para archivos individuales muy grandes
- **Memoria:** La versión `fork()` consume significativamente más memoria
- **Balanceamiento de carga:** La distribución de trabajo no considera el tamaño de los archivos

3.3.2. Mejoras Futuras

- Implementar paralelización a nivel de construcción del árbol de Huffman
- Desarrollar un balanceador de carga que considere el tamaño de archivos
- Optimizar el uso de memoria en la versión fork()
- Agregar soporte para compresión de directorios con subdirectorios recursivamente
- Mejorar el programa de usuarios para hacerlo más intuitivo

Capítulo 4

Conclusiones

1. **La paralelización es efectiva para datasets grandes:** Los resultados demuestran que la versión paralela con `fork()` logra una aceleración de 1.54x cuando se procesa un volumen significativo de datos como los 88 libros del Proyecto Gutenberg. Esto confirma que el overhead de creación de procesos se justifica cuando el trabajo computacional es suficientemente intensivo.
2. **La estrategia de paralelización por archivo es apropiada para este dominio:** El enfoque de asignar un proceso por archivo permite aprovechar eficientemente el paralelismo en sistemas multi-núcleo, ya que la compresión de diferentes archivos es naturalmente independiente y no requiere comunicación entre procesos durante la fase de procesamiento.
3. **La implementación sin pérdida mantiene integridad completa:** Las pruebas exhaustivas de verificación confirman que todas las versiones (serial, fork y pthread) producen resultados idénticos, demostrando que la paralelización no compromete la correctitud del algoritmo de Huffman y preserva la propiedad fundamental de compresión sin pérdida.

Referencias

- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*, 483–485.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.
- Gutenberg, P. (2024). *Free ebooks - project gutenberg*. Sitio web oficial. Descargado de <https://www.gutenberg.org/>
- Huffman, D. A. (1952). A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9), 1098–1101.
- IEEE, y Group, T. O. (2018). *The open group base specifications issue 7, 2018 edition*. Estándar POSIX.1-2017. Descargado de <https://pubs.opengroup.org/onlinepubs/9699919799/>
- Pacheco, P. (2011). *An introduction to parallel programming*. Morgan Kaufmann.
- Silberschatz, A., Galvin, P. B., y Gagne, G. (2018). *Operating system concepts* (10th ed.). John Wiley & Sons.
- Tanenbaum, A. S., y Bos, H. (2014). *Modern operating systems* (4th ed.). Prentice Hall.